

Desarrollo de Aplicaciones Web (DAW)

Desarrollo Web en Entorno Cliente — UT02 Sintaxis y Estructuras de Control de JavaScript

Instrucciones de Entrega y Buenas Prácticas

Para garantizar la calidad de los entregables y adquirir hábitos de desarrollo profesional, todos los estudiantes deben cumplir estrictamente las siguientes directrices:

- **Control de versiones con GitHub:** Es necesario enviar las modificaciones al repositorio de GitHub periódicamente registrando commits con regularidad.
- **Documentación de Código:** Documenta tu código JavaScript y HTML utilizando los comentarios correspondientes. Describe la lógica de las funciones, los tipos de variables y el flujo de los bloques condicionales e iterativos.
- **Uso del IDE:** Utiliza todas las funciones de autocompletado y validación de tu entorno de desarrollo (IDE) para agilizar el trabajo y evitar errores sintácticos.
- **Formateo de Código:** Formatea el código siempre antes de guardarlo con la función de formateo automático de tu IDE. La entrega de código desordenado o ilegible penalizará directamente en la nota de prácticas.
- **Estructura y Nombres de Archivos:** Crea la carpeta **“UT02/AC02.01_Actividad/”** dentro de tu repositorio. Asimismo, nombra los archivos como **“Actividad_?.html”** y/o **“Actividad_?.js”** (sustituyendo ?? por el identificador correspondiente).
- **IMPORTANTE — Conversión y Validación:** En todas las actividades prácticas que soliciten datos por pantalla con `prompt()`, es obligatorio realizar una conversión explícita del dato (por ejemplo, con `parseInt()` o `parseFloat()`) y verificar que la entrada es un número válido con `isNaN()` antes de operar.

Cuaderno de Actividades (Secuenciación Corrección UT02)

Actividad 01 — Identificadores válidos de variables (Investigativa/Teórica)

Indica razonadamente si los siguientes identificadores son nombres de variables válidos o incorrectos en JavaScript:

```
JavaScript
let $numero1;
let _$letra;
let 1numero;
let numero;1_123;
let $$$otroNumero;
let $_a__$4;
```

Actividad 02 — Cadenas de texto, caracteres de escape y comillas (Práctica)

Dadas las siguientes declaraciones de variables de tipo String:

Realiza los siguientes scripts:

```
JavaScript
let texto1 = "Una frase con 'comillas simples' dentro";
let texto2 = 'Una frase con "comillas dobles" dentro';
```

1. Realiza un script que muestre el contenido de ambas variables separado por un salto de línea (retorno de carro) en una ventana de alerta (alert()).
2. Realiza otro script que muestre el siguiente mensaje literal al usuario utilizando una sola función alert():

```
JavaScript
Hola Mundo!
Qué fácil es incluir 'comillas simples'
y "comillas dobles"
```

Actividad 03 — Propiedades del objeto Number y valores especiales (Práctica)

Desarrolla una función de JavaScript en un script HTML que contenga el siguiente código. Invócala adecuadamente desde el archivo para que los resultados se muestren en el navegador:

JavaScript

```
let maxValue = Number.MAX_VALUE;
let minValue = Number.MIN_VALUE;
alert("Max Value: " + maxValue);
alert("Min Value: " + minValue);
alert("Valor especial: " + (maxValue * 2));
```

Explica mediante comentarios en el código por qué se genera el último valor y qué representa conceptualmente.

Actividad 04 — Operadores lógicos con tipos no booleanos (Truthy/Falsy)

Prueba el siguiente script y explica detalladamente, mediante comentarios en cada línea de asignación de la variable 'vacío' y 'mensajeVacío', qué está ocurriendo y cómo reacciona JavaScript ante operadores lógicos aplicados a números y strings:

JavaScript

```
let cantidad = 0;
let vacío = !cantidad;
cantidad = 2;
vacío = !cantidad;

let mensaje = "";
let mensajeVacío = !mensaje;
mensaje = "Bienvenido";
mensajeVacío = !mensaje;
```

Actividad 05 — Comparaciones de igualdad débil frente a igualdad estricta (Teórico-Práctica)

Escribe un script JavaScript que implemente un código de prueba para analizar detalladamente la diferencia operativa entre los operadores de igualdad débil == e igualdad estricta ===, así como la desigualdad != y !==. Justifica el resultado mostrando cómo se aplican las conversiones de tipo implícitas frente a la comparación estricta de valor y tipo de dato.

Actividad 06 — Estructuras de decisión simples y validación de tipos (Práctica)

Desarrolla una función JavaScript que solicite dos números enteros al usuario utilizando la función `prompt()` y visualice en pantalla (mediante una ventana de alerta) cuál es el mayor de ellos.

Actividad 07 — Funciones, argumentos y toma de decisiones (Práctica)

Desarrolla una función JavaScript que reciba como argumento un único carácter (que previamente habrás solicitado al usuario mediante un `prompt()`). La función debe devolver `true` si el carácter recibido es una vocal y `false` en caso contrario. Desde tu código HTML principal, invoca a la función y muestra un mensaje comprensible informándole del resultado.

Actividad 08 — Bucles iterativos simples (Cálculo del Factorial) (Práctica)

El factorial de un número n ($n!$) es la multiplicación de todos los números enteros de 1 hasta n (ej: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$). Escribe un programa JavaScript que solicite un número entero positivo al usuario y calcule su factorial utilizando la estructura `for`. Implementa la lógica condicional para asegurar que el número introducido sea un entero válido y positivo.

Actividad 09 — Iteración condicional e informes por Consola (Práctica)

Desarrolla un código JavaScript que solicite un número entero al usuario. El script debe realizar una iteración desde 0 hasta el número proporcionado, evaluando paso a paso si el número del ciclo es par o impar. Muestra este informe en la consola de depuración del navegador web mediante `console.log()`.

Actividad 10 — Lógica condicional e iterativa anidada (Patrón de Asteriscos)

Crea un programa JavaScript que solicite un número entero al usuario y visualice en la página HTML un triángulo alineado a la izquierda usando asteriscos. Si el usuario introduce el número 5, el resultado visual en la página web debe ser exactamente el siguiente:

Utiliza bucles anidados en la implementación de la lógica.

JavaScript

```
*  
**  
***  
****  
*****
```

Actividad 11 — Algoritmia avanzada (Cálculo de Números Primos en Consola)

Desarrolla una script JavaScript con una función encargada de calcular y encontrar todos los números primos comprendidos entre 1 y 100. Muestra la lista final de números primos identificados a través de la consola del navegador mediante `console.log()`. Guarda la lógica del script en un archivo JavaScript externo e impórtalo en tu documento HTML.

Actividad 12 — Depuración práctica y ámbitos de variables con el Debugger

Descarga el archivo comprimido `depurar.zip` provisto en la plataforma. Abre el proyecto en tu navegador web y activa las Herramientas de Desarrollo (F12). Utiliza la pestaña Sources (Fuentes) para colocar puntos de interrupción (breakpoints) dentro del script. Inspecciona visualmente el valor y el alcance de las variables `sumaLocal` y `sumaGlobal` para comprender el comportamiento de los ámbitos local y global en tiempo real. Escribe una breve memoria explicativa con tus hallazgos.